

SIMATS ENGINEERING

ASSESSMENT TOOL 1 : Concept Mapping

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1518

COURSE OUTCOME COVERED

CO1: *Apply cloud computing technologies including hardware-software evolution, virtualization, web services, and service models to develop cloud-based solutions. (BL2)*

Assessment Tool Weightage: 20%

Dave's Taxonomy: Imitation (Level 1)
Question Count: 4

Total Marks: 50

Code of Conduct

I G. Jaswanth - 192472235 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. Jaswanth

Assessment Tasks

Concept Mapping Tasks

Create concept maps for the following tasks. Each map must show key nodes, link labels, and a logical hierarchy.

1. Concept Map 1: Cloud Computing Fundamentals. Include definition, characteristics, benefits, and limitations.
2. Concept Map 2: Evolution of Cloud Computing. Connect hardware evolution, internet software evolution, distributed systems, and service delivery.
3. Concept Map 3: Service Models. Map IaaS, PaaS, SaaS, and XaaS with provider responsibilities and user responsibilities.
4. Concept Map 4: Virtualization and Web Services as Enablers of Cloud. Show how these two concepts support cloud deployment and scalability.

Expected concept nodes:

- Elasticity, scalability, resource pooling, on-demand access, measured service
- Virtual machine, hypervisor, abstraction, API, SOAP, REST, provider, consumer
- Infrastructure layer, platform layer, application layer, shared responsibility

Concept Mapping Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Identification	25%	All major concepts are accurately identified	Most concepts are identified correctly	Limited concepts are identified	Essential concepts are missing
Relationship Mapping	25%	Links between concepts are accurate and meaningful	Most links are correct	Some links are weak or unclear	Relationships are mostly incorrect

Hierarchy and Organization	20%	Excellent structure with clear hierarchy	Mostly organized with minor issues	Basic structure with weak hierarchy	No logical organization
Technical Relevance	20%	Map strongly reflects cloud course content	Mostly aligned to topic	Partially aligned to topic	Weak alignment to topic
Visual Clarity	10%	Neat, readable, and easy to interpret	Generally readable	Somewhat cluttered	Poorly presented

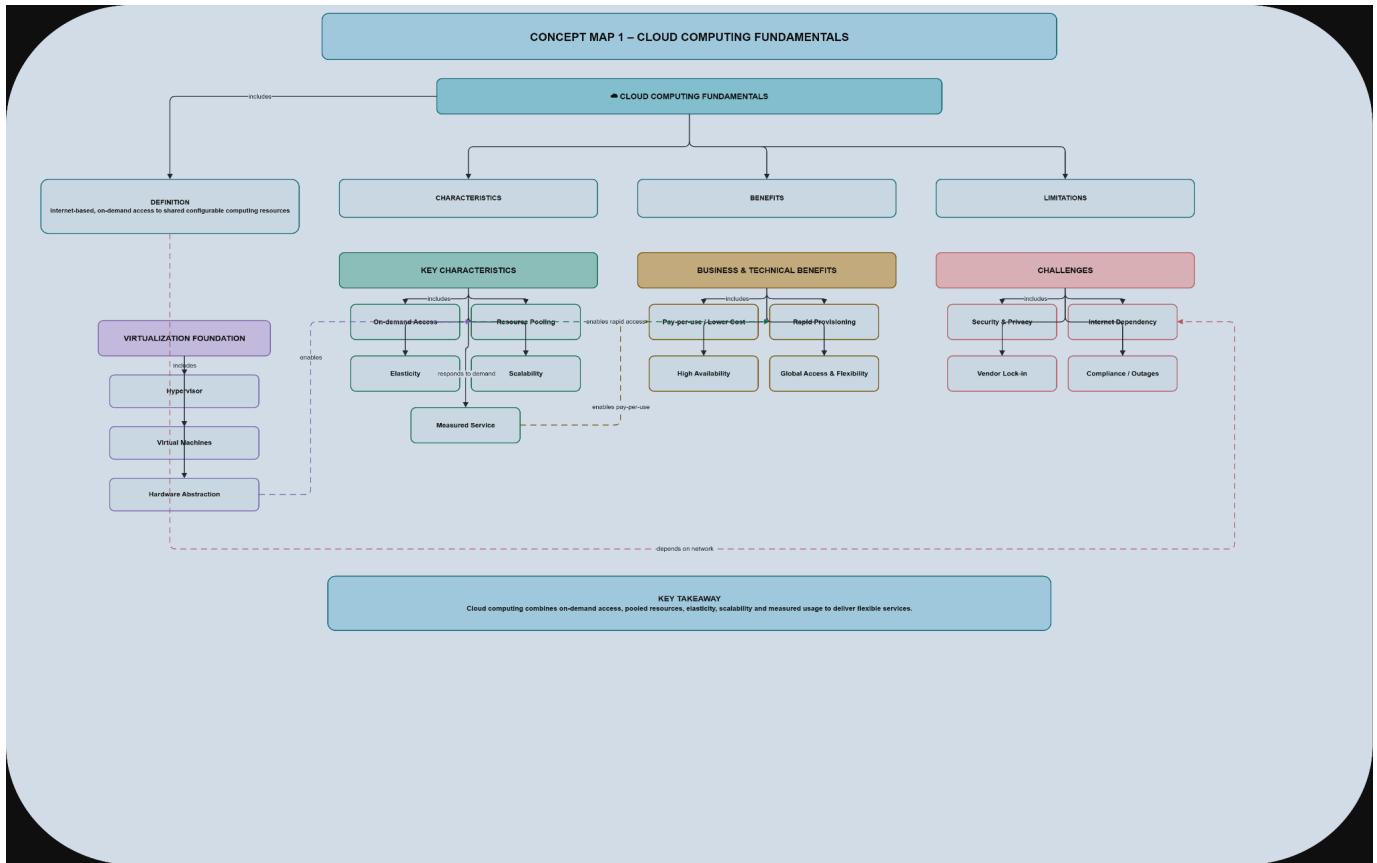


Figure 1: Concept Map – Cloud Computing Fundamentals

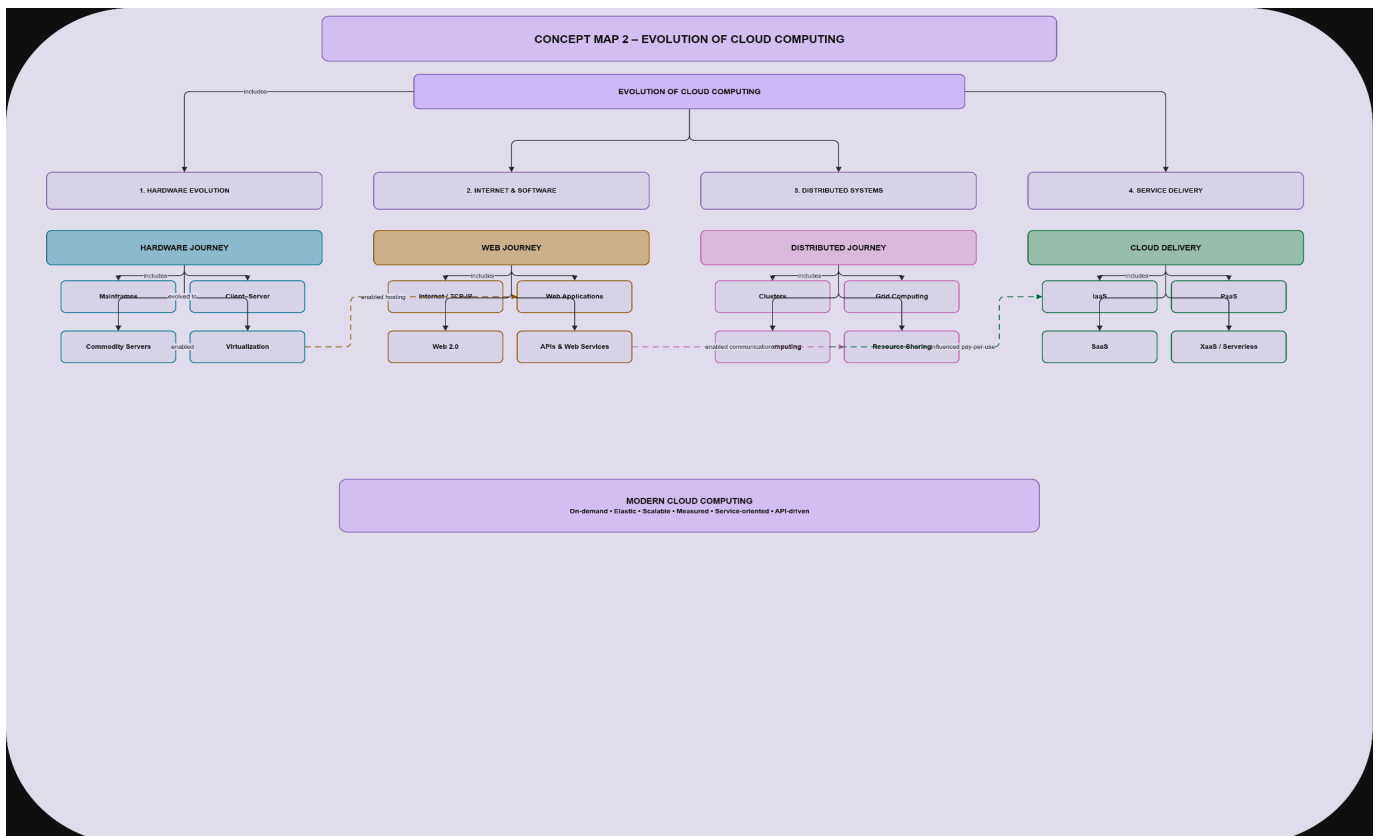


Figure 2: Concept Map – Evolution of Cloud Computing

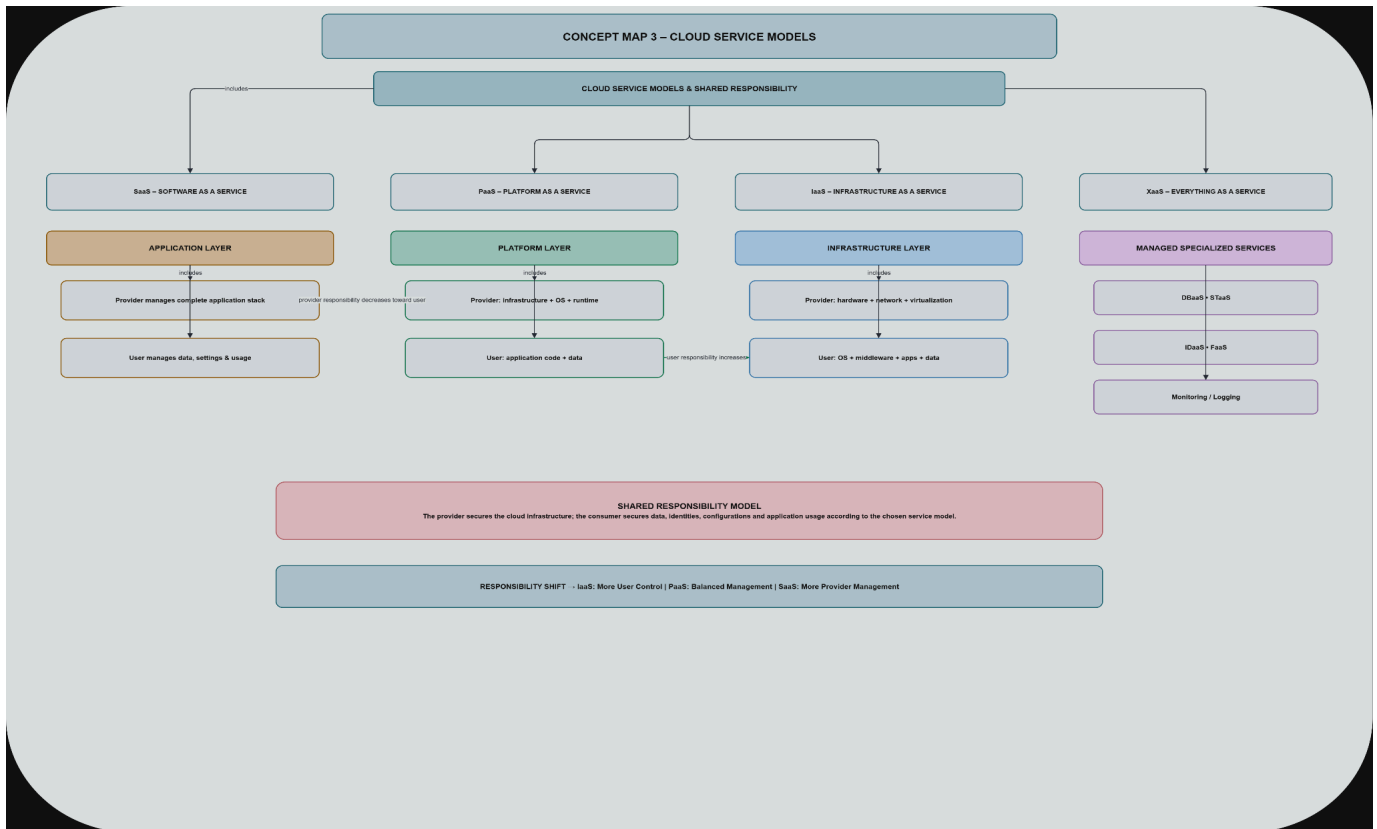


Figure 3: Concept Map – Cloud Service Models

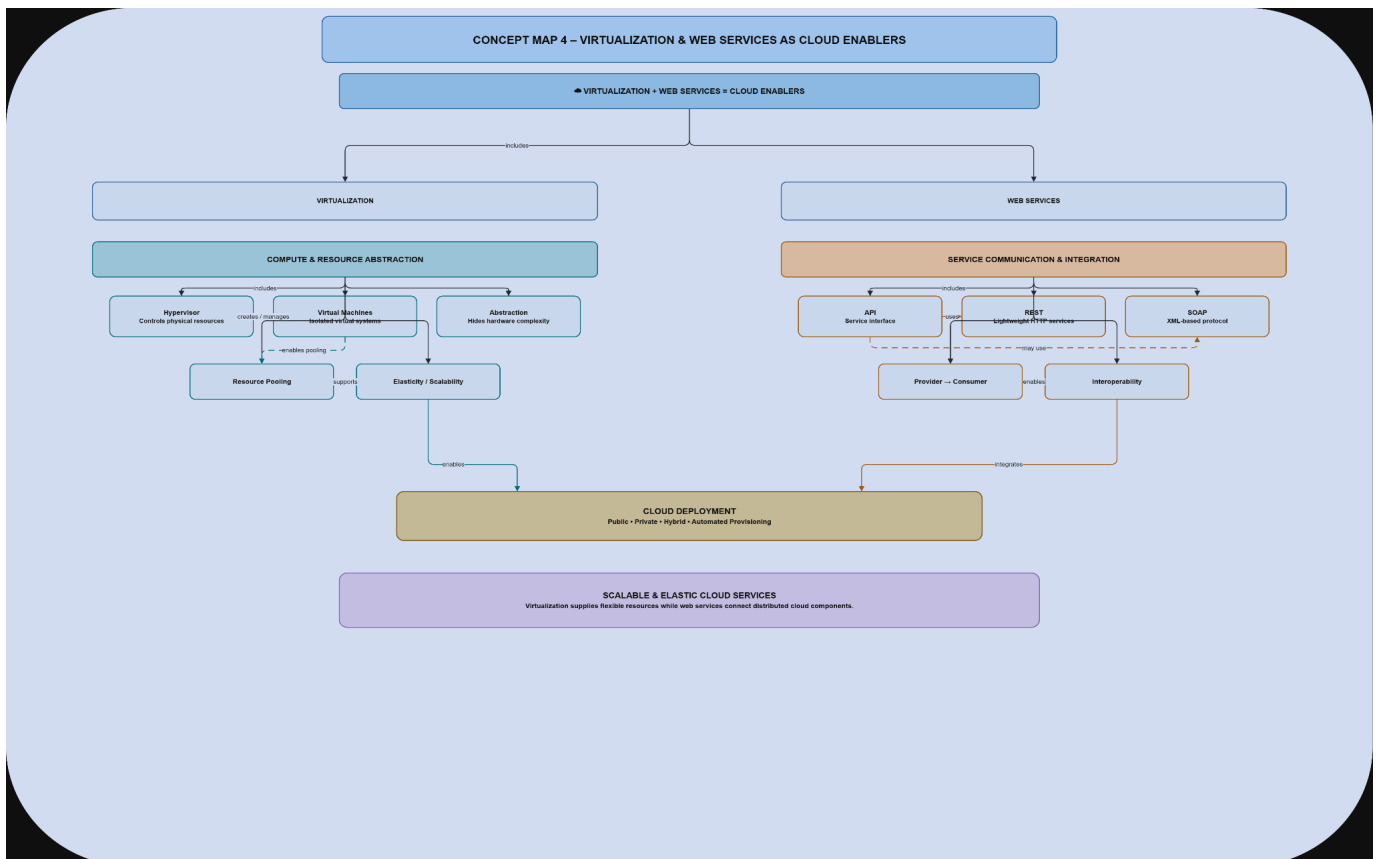
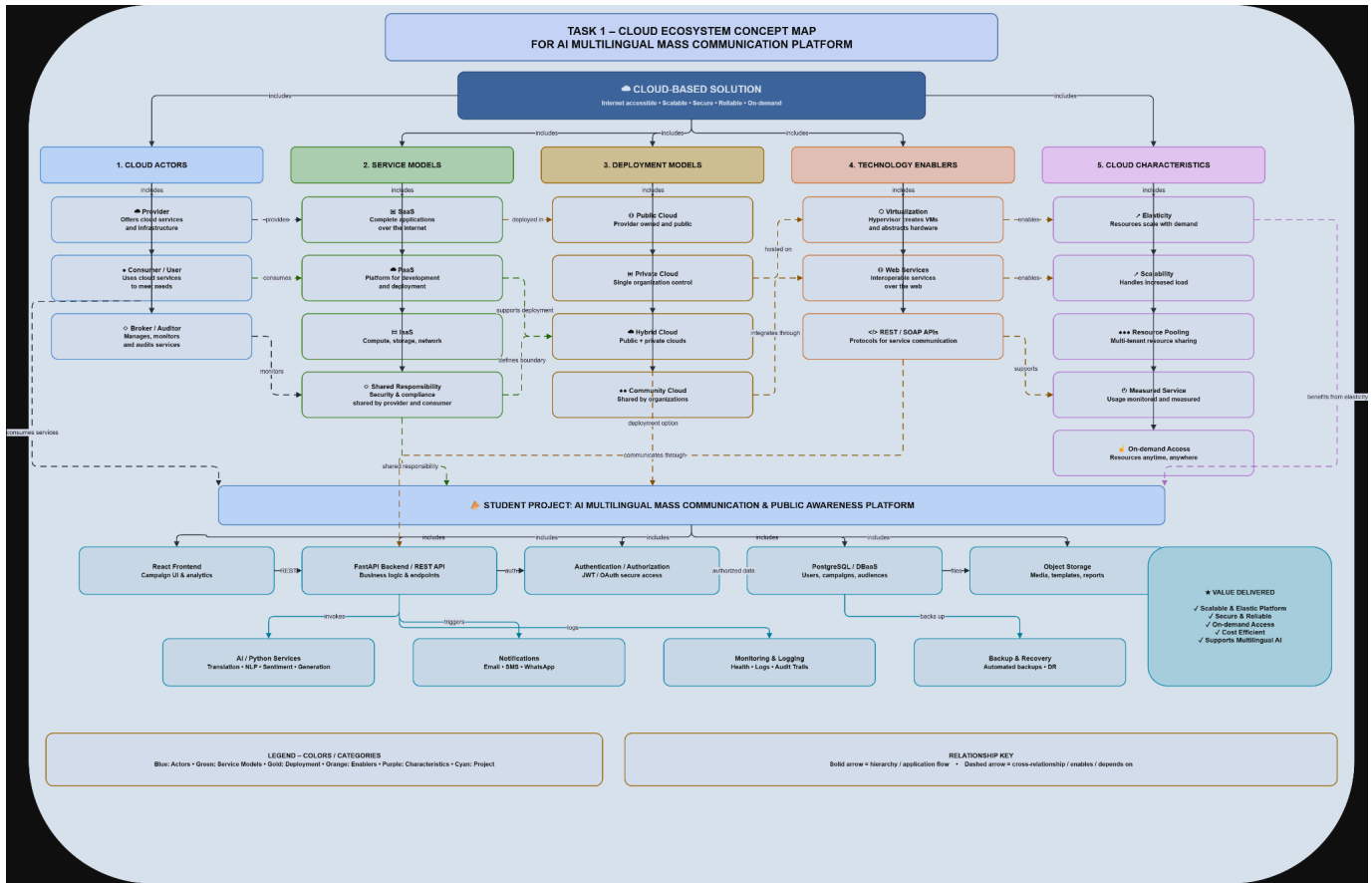


Figure 4: Concept Map – Virtualization and Web Services as Cloud Enablers



The concept map connects the major elements of the cloud ecosystem with the proposed AI Multilingual Mass Communication and Public Awareness Platform. It represents cloud actors, service models, deployment models, virtualization, web services, REST/SOAP APIs, elasticity, scalability, resource pooling, measured services, and on-demand access. The project uses a React frontend, FastAPI REST backend, authentication services, managed database, object storage, AI/Python services, notification services, monitoring, logging, and backup mechanisms. These components demonstrate how cloud technologies can be integrated to build a scalable, secure, and service-oriented application.

Task 2A: Service Model Responsibility Matrix

The following matrix compares major cloud service categories based on the responsibilities of the cloud provider and the student application team. The responsibility split is evaluated with respect to the **AI Multilingual Mass Communication & Public Awareness Platform**.

Service Category	Cloud Provider Manages	Student Application Team Manages	Why It Fits the Project
SaaS	Complete application, infrastructure, platform, maintenance, updates and service availability	User configuration, access settings, organizational data and proper usage	Useful for ready-to-use communication or collaboration services without maintaining the complete software stack

PaaS	Physical servers, networking, virtualization, operating system, runtime and platform availability	FastAPI backend code, REST APIs, application configuration, business logic and data	Suitable for deploying the backend while allowing the team to focus mainly on application development
IaaS	Physical servers, storage hardware, networking and virtualization layer	Virtual machine OS, middleware, runtime, applications, security configuration and data	Useful when greater control over the computing environment and infrastructure configuration is required
FaaS / Serverless	Runtime infrastructure, provisioning, execution environment, availability and automatic scaling	Functions, event logic, permissions, triggers and application configuration	Suitable for notification triggers, scheduled operations and background tasks that execute only when required
Database-as-a-Service (DBaaS)	Database infrastructure, database engine availability, maintenance, patching and managed backup facilities	Database schema, tables, queries, indexing, application data and access rules	Suitable for storing users, audiences, campaigns, templates and message-delivery information
Storage-as-a-Service (STaaS)	Storage infrastructure, capacity, availability, durability and hardware management	Files, campaign media, permissions, lifecycle policies and data organization	Suitable for storing campaign images, documents, generated reports, templates and other unstructured files
Identity-as-a-Service (IDaaS)	Authentication infrastructure, identity platform, authentication mechanisms and service availability	User roles, authorization policies, account integration and application access rules	Provides secure authentication and role-based access for administrators and campaign managers
Monitoring / Logging Services	Monitoring infrastructure, log collection platform, storage infrastructure and service availability	Logging strategy, dashboards, alerts, retention requirements and incident response	Helps detect API errors, application failures, security events and performance problems

Responsibility Analysis:

The matrix demonstrates the **shared responsibility model** in cloud computing. In IaaS, the application team has greater control but also greater management responsibility. With PaaS, SaaS and specialized managed services, more infrastructure responsibilities are handled by the cloud provider. However, the application team remains responsible for application logic, data, user authorization, configurations and secure usage. For the **AI Multilingual Mass Communication & Public Awareness Platform**, managed cloud services reduce infrastructure-management overhead while improving scalability, reliability and maintainability.

Task 2B: Cloud Service Decision Matrix

Project Module	Selected Cloud Service	Service Model / Type	Role in the Project	Justification
Authentication	Managed Identity & Authentication Service	IDaaS	Authenticates administrators and campaign managers and controls access to protected features	Provides centralized and secure authentication while reducing the need to develop authentication infrastructure from scratch
Frontend Hosting	Static Web Hosting + CDN	PaaS / Managed Hosting	Hosts and delivers the React frontend to users	CDN-based delivery improves loading speed, availability and scalability during high traffic
Backend / API Layer	Managed Compute Platform	PaaS	Runs the FastAPI backend, REST APIs and application business logic	Reduces server-management work and allows the backend to scale horizontally according to API traffic
Database	Managed PostgreSQL	DBaaS	Stores users, audience information, campaigns, templates and delivery records	Provides reliable relational storage while the provider handles much of the database infrastructure and maintenance

Object Storage	Cloud Object Storage	STaaS	Stores campaign images, documents, templates, generated reports and other media	Provides scalable and durable storage for large unstructured files
Analytics / Logging	Managed Logging & Analytics Service	Monitoring-as-a-Service	Collects API logs, campaign events, errors and operational information	Centralized logs simplify troubleshooting, auditing and analysis of application behaviour
Notification Service	Serverless Functions + Messaging Service	FaaS	Processes email, SMS, WhatsApp and other campaign notification events	Event-driven processing prevents notification workloads from blocking the main API and can scale during large campaigns
AI / Python Module	Managed Container / Compute Service	PaaS	Executes translation, NLP, sentiment analysis, content generation and personalization modules	AI workloads can scale independently from the main backend and receive additional compute resources when required
Security	IAM + Secrets Management + TLS/WAF	Security-as-a-Service	Protects identities, API credentials, network traffic and cloud resources	Supports least-privilege access, encrypted communication and secure credential management
CI/CD	Managed CI/CD Pipeline	DevOps-as-a-Service	Automatically builds, tests and deploys frontend and backend updates	Provides repeatable deployments and reduces errors caused by manual releases
Backup & Recovery	Managed Backup + Cloud Storage	Backup-as-a-Service	Maintains backups of databases and important application data	Enables recovery from accidental deletion, corruption or service failures

Monitoring	Cloud Monitoring & Alerting	Monitoring-as-a-Service	Tracks availability, latency, failures, resource utilization and service health	Allows the team to detect problems early and make scaling or maintenance decisions
------------	-----------------------------	-------------------------	---	--

Service Selection Justification:

The cloud services are selected according to the actual requirements of the **AI Multilingual Mass Communication & Public Awareness Platform** rather than using every available cloud service. Managed identity secures user access, PaaS supports the FastAPI backend, DBaaS manages structured application data, and object storage handles campaign media. Serverless notification processing and independently scalable AI services are suitable for workloads whose demand can change significantly during large public-awareness campaigns. Monitoring, logging, security, backup and CI/CD improve the reliability, maintainability and security of the overall system.

Task 3: API Flow, Elasticity, and Scalability Design

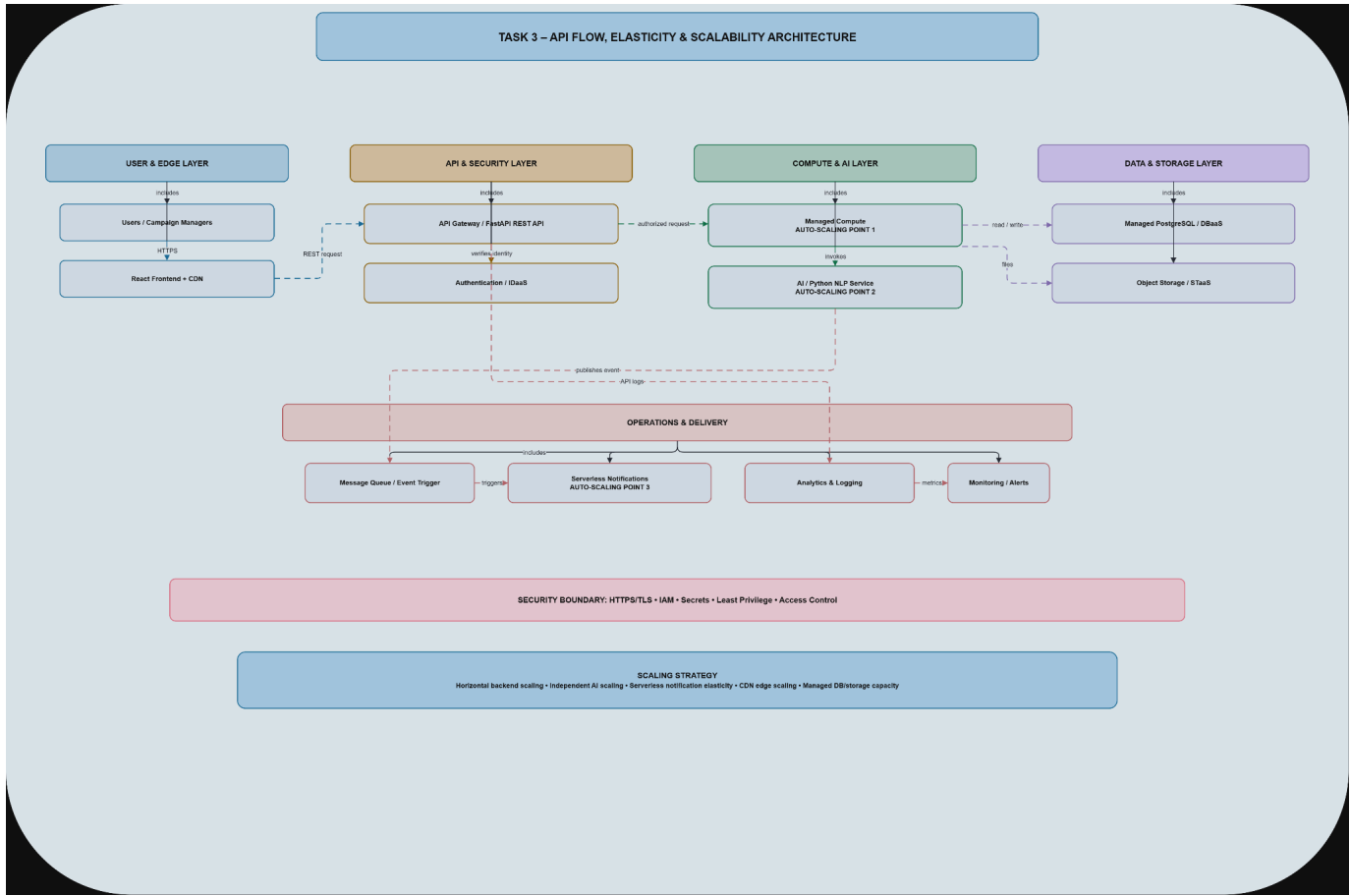


Figure Title:

Cloud API Flow, Elasticity and Scalability Architecture for AI Multilingual Mass Communication & Public Awareness Platform

Architecture Explanation

The proposed architecture follows a scalable cloud-based design in which the React web application acts as the user interface for administrators and campaign managers. The frontend communicates securely with the FastAPI backend through REST APIs over HTTPS.

The backend verifies users through the authentication and authorization service before allowing access to protected operations. After authentication, the compute layer handles campaign creation, audience management, content processing and other application logic.

The AI/Python module provides services such as multilingual translation, NLP-based content processing, sentiment analysis and personalization. Structured information such as users, campaigns, audiences and delivery records is stored in a managed PostgreSQL database, while campaign images, documents, templates and generated files are maintained in cloud object storage.

Notification requests can be passed through a message queue/event system and processed asynchronously by serverless notification services. This allows email, SMS and WhatsApp workloads to operate independently without blocking normal API requests.

API Flow

User → React Frontend → REST API / FastAPI Backend → Authentication → Compute Layer → Database / Object Storage / AI Services → Notification Services → User

REST APIs provide a standardized communication mechanism between the frontend, backend and cloud services. This loose coupling makes individual modules easier to maintain, replace and scale.

Elasticity and Scalability

The architecture contains multiple scaling points:

1. Backend/API Scaling: Multiple backend instances can be created when API traffic increases.
2. AI/Python Scaling: AI workloads can scale independently because translation and NLP operations may require more computational resources.
3. Notification Scaling: Serverless notification workers can automatically scale during large campaign distributions.
4. Storage and Database Scaling: Managed database and object-storage services can increase capacity as application data grows.
5. Frontend Scaling: CDN-based frontend delivery distributes static content efficiently to a large number of users.

Elasticity allows resources to increase or decrease dynamically according to current demand, whereas scalability allows the architecture to support long-term growth in users, campaigns, data and message volume.

Security and Monitoring

The architecture uses HTTPS/TLS, IAM, authentication, authorization, secrets management and least-privilege access controls to establish secure boundaries between cloud components. Monitoring and

logging services collect API logs, errors, resource utilization and performance metrics. Alerts can be generated when abnormal behaviour or service failures are detected.

Task 3 Conclusion

The architecture demonstrates how APIs, managed cloud services, elasticity and scalability work together to create a reliable cloud-based solution. Separating the frontend, backend, AI processing, data storage and notification components enables each service to scale independently and improves performance, availability, maintainability and user experience.

Task 4: Technical Justification

The proposed **AI Multilingual Mass Communication & Public Awareness Platform** demonstrates the application of cloud computing concepts covered under **CO1**. The concept maps, responsibility matrices, service decision matrix, and architecture diagram collectively show how service models, web services, virtualization, elasticity, scalability, and cloud responsibility boundaries can be applied to develop a practical cloud-based solution.

1. Service Model Selection

The architecture uses a combination of **PaaS, DBaaS, STaaS, IDaaS, FaaS/serverless, and managed monitoring services**. PaaS or managed compute is suitable for the FastAPI backend because the cloud provider manages the underlying infrastructure, operating environment, and platform while the application team focuses on APIs and business logic. DBaaS manages database infrastructure, object storage handles unstructured campaign files, and IDaaS provides managed authentication.

This combination reduces infrastructure-management overhead while retaining control over application functionality and data.

2. Role of APIs and Web Services

REST APIs provide the primary communication mechanism between the React frontend, FastAPI backend, AI/Python services, database-related operations, and other cloud services.

API-based communication creates **loose coupling**, allowing individual components to be developed, maintained, deployed, and scaled independently. Web-service technologies also make it easier to integrate external services such as email, SMS, WhatsApp, translation, and notification providers.

3. Elasticity

The workload of a public-awareness platform may change significantly during major campaigns. **Elasticity** allows cloud resources to automatically increase when demand rises and decrease when demand falls.

For example, backend instances, AI-processing services, and serverless notification workers can obtain additional resources during high-volume campaigns. This improves performance while avoiding the cost of permanently maintaining maximum capacity.

4. Scalability

Scalability enables the system to support long-term growth in users, campaigns, audience records, stored media, API requests, and notification volume.

The architecture separates the frontend, backend, AI processing, database, object storage, and notification services so that individual components can scale according to their own workload. Horizontal scaling of backend services and managed storage/database capacity helps maintain performance as usage increases.

5. Virtualization and Managed Compute

Virtualization provides an abstraction between physical hardware and application workloads through technologies such as **virtual machines and hypervisors**. Modern managed compute and container platforms build on similar abstraction principles to provide isolated and flexible computing environments.

Using managed compute allows the project to obtain computing resources without directly managing physical servers. This improves resource utilization, deployment flexibility, isolation, and scalability.

6. Cloud Responsibility Boundaries

The architecture follows the **shared responsibility model**.

The **cloud provider** is responsible for areas such as physical infrastructure, networking hardware, virtualization, and the managed portions of selected cloud services.

The **student application team** remains responsible for:

- Application code and business logic
- User roles and authorization
- Application data
- API security
- Cloud-service configuration
- Secrets and credentials
- Monitoring requirements
- Secure use of cloud resources

The exact responsibility split changes according to whether IaaS, PaaS, SaaS, or another managed service is selected.

CO1 Justification

Therefore, the proposed architecture satisfies **CO1** by applying cloud computing technologies—including **virtualization, web services, APIs, service models, managed cloud resources, elasticity, scalability,**

and shared responsibility—to design a practical cloud-based solution. The architecture demonstrates not only the selection of cloud technologies but also how those technologies interact to provide a **scalable, secure, reliable, and maintainable application**.

Analytical Question 1

1. Evaluate the effectiveness of your concept map in representing cloud computing principles. Which design decisions most strongly support your solution, and why?

Answer:

The concept map effectively represents cloud computing principles by connecting the major concepts of **cloud actors, service models, deployment models, virtualization, web services, APIs, elasticity, scalability, resource pooling, and measured services** with the proposed **AI Multilingual Mass Communication & Public Awareness Platform**.

One of the strongest design decisions is the use of **managed cloud services** for different project modules. The FastAPI backend can use a managed compute or PaaS environment, PostgreSQL can be provided through DBaaS, campaign media can use object storage, and authentication can use IDaaS. This reduces infrastructure-management effort while allowing the application team to focus on application development.

Another important decision is the **separation of major components** such as the React frontend, FastAPI backend, AI/Python services, database, object storage, and notification services. This modular architecture allows individual components to be maintained and scaled independently.

The concept map also represents the **shared responsibility model**, showing that the cloud provider manages underlying infrastructure and managed-service components, while the application team remains responsible for application code, data, authorization, configuration, and secure usage.

Therefore, the concept map is effective because it connects theoretical cloud principles with their **practical implementation in the project**, clearly demonstrating how cloud technologies contribute to scalability, flexibility, reliability, and efficient resource utilization.

Analytical Question 2

2. Analyze how scalability, elasticity, and resource management influence the architecture shown in your concept map. What challenges could arise if these capabilities were absent?

Answer:

Scalability, elasticity, and resource management are important in the proposed **AI Multilingual Mass Communication & Public Awareness Platform** because the number of users, campaigns, API requests, AI-processing tasks, and notifications can vary significantly.

Scalability allows the system to handle long-term growth by increasing resources or adding additional service instances. For example, the FastAPI backend can use horizontal scaling to handle more API requests, while the database and object storage can increase capacity as campaign and audience data grows.

Elasticity allows resources to dynamically increase or decrease according to current demand. During a large public-awareness campaign, additional backend, AI-processing, and notification resources can be allocated. When demand decreases, unnecessary resources can be released, improving cost efficiency.

Resource management ensures that compute, storage, memory, and network resources are efficiently allocated among different cloud workloads. Virtualization, resource pooling, and managed cloud services help achieve this without requiring the application team to manage physical hardware directly.

If these capabilities were absent, the system could experience **slow response times, server overload, notification delays, AI-processing bottlenecks, storage limitations, and service failures** during periods of high demand. Maintaining fixed infrastructure for maximum possible traffic would also increase cost and lead to resource wastage during low-demand periods.

Therefore, scalability, elasticity, and efficient resource management make the architecture **responsive, cost-efficient, reliable, and capable of supporting both sudden traffic increases and long-term application growth**.

Analytical Question 3

3. Justify the selection of the APIs, web services, or cloud services included in your concept map. How do they contribute to the overall functionality and efficiency of the system?

Answer:

The APIs, web services, and cloud services were selected according to the requirements of the **AI Multilingual Mass Communication & Public Awareness Platform**.

REST APIs are used as the primary communication mechanism between the React frontend and FastAPI backend. REST is lightweight, widely supported, and allows different application components to communicate using standard HTTP methods. It also creates loose coupling between the frontend and backend.

Managed authentication or IDaaS provides secure user authentication and supports role-based access for administrators and campaign managers. **DBaaS with PostgreSQL** is suitable for structured information such as users, campaigns, audiences, templates, and delivery records. **Object storage** is used for unstructured data such as campaign images, documents, and generated reports.

The **AI/Python service** handles computational tasks such as multilingual translation, NLP, sentiment analysis, content generation, and personalization. Keeping AI processing separate from the main backend allows it to scale independently.

FaaS/serverless and messaging services are appropriate for asynchronous notification tasks such as email, SMS, and WhatsApp delivery. This prevents large notification workloads from blocking normal API requests.

Finally, **monitoring and logging services** provide information about API errors, system performance, resource utilization, and application failures.

Therefore, these services improve the system's **modularity, scalability, security, maintainability, and efficiency**. Each project requirement is mapped to an appropriate cloud service rather than placing all functionality on a single server.

Analytical Question 4

4. Critically examine the relationships between the components in your concept map. How do these interactions improve reliability, performance, or user experience?

Answer:

The components in the concept map are designed to interact through a **modular and service-oriented architecture**. Each component performs a specific function while communicating with other components through APIs and managed cloud services.

The **React frontend** communicates with the **FastAPI backend through REST APIs**, preventing users from directly accessing databases or internal services. The backend interacts with the **authentication service** to verify user identity and permissions before processing protected requests, improving security and reliability.

The backend communicates with **PostgreSQL** for structured data and **object storage** for campaign media and files. Separating these storage mechanisms improves efficiency because each type of data is handled by a service designed for its requirements.

The **AI/Python module** operates separately from the main backend for translation, NLP, sentiment analysis, and personalization. This separation allows computationally intensive AI workloads to scale independently without significantly affecting normal API performance.

Similarly, notifications can be processed through **message queues and serverless workers**. Email, SMS, and WhatsApp delivery can therefore occur asynchronously without forcing users to wait for the complete distribution process.

Finally, **monitoring and logging services** collect performance metrics, errors, and operational events. This enables faster detection of failures and helps maintain system availability.

These interactions improve **reliability through component isolation, performance through independent scaling and asynchronous processing, and user experience through faster response times and consistent service availability**.

Analytical Question 5

5. Reflect on the process of developing your concept map. How has creating and analyzing the map changed your understanding of cloud computing architecture, and what improvements would you make if you were redesigning it?

Answer:

Developing the concept map improved my understanding of how individual cloud computing concepts work together to form a complete architecture. Initially, concepts such as **virtualization, service models, web services, elasticity, scalability, resource pooling, and shared responsibility** can appear independent. Creating the map showed how these concepts are interconnected in a practical cloud solution.

Mapping the **AI Multilingual Mass Communication & Public Awareness Platform** also helped me understand that selecting a cloud service is not only about hosting an application. Different components have different requirements. For example, DBaaS is appropriate for structured application data, object storage is suitable for media files, managed compute can host the backend, and serverless services are useful for event-driven notification workloads.

The activity also clarified the **shared responsibility model**. Using managed cloud services reduces infrastructure-management responsibilities, but the application team still remains responsible for areas such as application code, data, authorization, configuration, and secure service usage.

If I redesigned the architecture, I would improve it by adding **load balancing, caching, multi-zone availability, disaster recovery, queue retry mechanisms, automated scaling policies, and cost monitoring**. I would also perform load testing to determine appropriate scaling thresholds for the backend, AI services, and notification components.

Overall, creating and analyzing the concept map changed my understanding of cloud computing from viewing it mainly as **remote infrastructure** to understanding it as an ecosystem of interconnected, scalable, managed, and service-oriented technologies used to build reliable applications.

CO1 Reflection

Through this assessment, I developed a clearer understanding of how fundamental cloud computing concepts are applied while designing a practical cloud-based application. The concept maps helped me connect **hardware and software evolution, virtualization, web services, service models, elasticity, scalability, resource pooling, and measured services** instead of viewing them as separate concepts.

Designing the **AI Multilingual Mass Communication & Public Awareness Platform** helped me understand how different cloud service models can be selected according to application requirements. Managed compute can support the FastAPI backend, DBaaS can manage structured application data, object storage can store campaign media, IDaaS can provide authentication, and serverless services can support event-driven notification processing.

The architecture also helped me understand the importance of **REST APIs and web services** for communication between distributed components. Virtualization and managed computing provide abstraction from physical infrastructure, while elasticity and scalability allow applications to respond efficiently to changing workloads.

Another important learning outcome was understanding the **shared responsibility model**. Although cloud providers manage much of the underlying infrastructure, the application team continues to be responsible for application code, data, authorization, configuration, and secure use of cloud resources.

Overall, this assessment demonstrated how cloud technologies can be combined to design a **scalable, secure, reliable, and maintainable cloud-based solution**, thereby satisfying the requirements of **CO1**.